SMART POTHOLE DETECTION SYSTEM

v2.0 — Dual LiDAR Depth Sensing Edition

TF-Luna (UART/I2C) + Benewake TF02-i CAN | HC-SR04 Fallback | IoT + AI/ML + Cloud Dashboard

Primary Depth	Secondary Depth	Backup Depth	Fusion Logic
TF-Luna (UART/I2C)0.2m–8m | ±2mm250Hz update rate	Benewake TF02-i CAN0.1m–22m | ±1%Long-range precision	HC-SR04 Ultrasonic2cm–4m | ±3mmActivates on LiDAR fail	Weighted average when both LiDARs active. Auto-failover to HC-SR04 if both offline.

1. System Overview & Architecture

This blueprint describes a full production-grade pothole detection system using a Raspberry Pi 4 as the edge brain, a TF-Luna LiDAR as the primary close-range depth sensor, a Benewake TF02-i CAN LiDAR as the long-range / high-precision secondary sensor, and an HC-SR04 ultrasonic as an automatic fallback. All three sensors feed a sensor-fusion layer that picks the most reliable reading at every moment. Detected potholes — with GPS coordinates, measured depth, estimated width, and AI severity score — are streamed over MQTT to a cloud backend and displayed on a live government web dashboard.

1.1 Four-Layer Architecture

Layer	Key Components	Responsibility	Protocol
Layer 1: Edge Device	Raspberry Pi 4B, MPU-6050, TF-Luna, TF02-i, HC-SR04, NEO-M8N GPS, Camera	Detect, measure, infer on-device	I2C / UART / SPI / GPIO
Layer 2: AI/ML Engine	YOLOv8n TFLite + Regression CNN, runs on Pi	Classify pothole, estimate width from camera + fused depth	Local TFLite inference
Layer 3: Cloud Backend	FastAPI, Mosquitto MQTT, PostgreSQL+PostGIS	Ingest, store, serve, push WebSocket events	MQTT + REST + WebSocket
Layer 4: Dashboard	React.js, Leaflet, Admin Portal	Live map, alerts, govt repair tracking	HTTPS + WSS

1.2 Depth Sensor Priority Logic

Priority	Sensor	Condition	Notes
1	TF-Luna (UART/I2C)	Signal valid, distance < 6m	Primary sensor. Fast 250Hz, ±2mm, perfect for close-range pothole profiling
2	TF02-i (CAN Bus)	Signal valid, distance < 20m	Secondary / long-range. ±1% accuracy. Used when TF-Luna saturates (deep/wide potholes)
3	HC-SR04 (GPIO)	Both LiDARs offline/error	Automatic fallback. Triggers alert on dashboard: 'Fallback sensor active'
Fusion	Weighted Average	Both LiDARs valid	(0.6 × TF-Luna) + (0.4 × TF02-i). TF-Luna weighted higher for short-range accuracy

2. Complete Hardware Requirements

2.1 Core Processing Unit

Component	Model / Spec	Est. Cost (INR)	Purpose
Single Board Computer	Raspberry Pi 4B — 4GB RAM	Rs. 5,000–7,000	Main compute: runs sensors, AI model, MQTT client
MicroSD	64GB Class-10 / A2 rated	Rs. 500–700	OS + TFLite models + data buffer
Vehicle Power	12V→5V/3A DC-DC Buck converter	Rs. 200–400	Powers Pi from bike/car 12V rail cleanly
UPS / Battery backup	UPS HAT for Pi (18650 cells)	Rs. 800–1,200	Prevents data loss on engine-off jolt

2.2 Depth Sensing — LiDAR Sensors

Sensor	Interface	Cost (INR)	Specifications
Benewake TF-Luna	UART (default) or I2C	Rs. 2,500–3,500	Range: 0.2m–8m | Accuracy: ±2mm | Update: 250Hz | FOV: 2° | Supply: 3.3–5V
Benewake TF02-i CAN	CAN Bus (MCP2515 needed)	Rs. 6,000–9,000	Range: 0.1m–22m | Accuracy: ±1% | Update: 100Hz | Industrial grade, IP67 rated
HC-SR04 (Backup)	GPIO (TRIG/ECHO)	Rs. 150–300	Range: 2cm–4m | Accuracy: ±3mm | 40Hz | Activates only when both LiDARs offline

2.3 Remaining Sensors & Modules

Component	Model	Cost (INR)	Purpose
IMU	MPU-6050 (6-DOF)	Rs. 150–300	Primary pothole trigger via Z-axis jolt detection
GPS	NEO-M8N (10Hz, high accuracy)	Rs. 900–1,500	Precise lat/long for every pothole event
Camera	Raspberry Pi Camera v2 or OV5647	Rs. 1,200–2,500	Frame capture for YOLOv8 visual detection + width estimation
CAN Interface	MCP2515 SPI-to-CAN module	Rs. 300–600	Required to connect TF02-i CAN bus to Raspberry Pi SPI
4G LTE Module	SIM7600 / Huawei E3372	Rs. 2,000–3,500	Cloud uplink on roads without Wi-Fi

Component	Model	Cost (INR)	Purpose
Enclosure	IP67 ABS box with cable glands	Rs. 500–900	Waterproof protection for all electronics
Buzzer + LED	5V Active buzzer + RGB LED	Rs. 80–150	Local driver alert on detection

2.4 Total Budget

Configuration	Estimated Cost (INR)
Prototype: Pi + TF-Luna + HC-SR04 backup + MPU-6050 + GPS + Camera	Rs. 12,000 – 17,000
Full Production: Adds TF02-i CAN + MCP2515 + 4G + IP67 enclosure + UPS HAT	Rs. 22,000 – 32,000
Fleet Unit (bulk, 20% discount)	Rs. 18,000 – 26,000 per unit

3. Wiring & Connection Diagrams

3.1 TF-Luna LiDAR — UART Mode (Recommended)

UART mode is recommended for TF-Luna because it gives 250Hz update rate with minimal CPU overhead. Connect as follows:

TF-Luna Pin	Pi GPIO Pin	Pi Header #	Notes
VCC (5V)	5V Power	Pin 2 or 4	Use 5V rail — do NOT use 3.3V, sensor requires 5V
GND	Ground	Pin 6	Common ground
TX (TF-Luna out)	GPIO 15 / RX	Pin 10	TF-Luna sends data TO Pi RX
RX (TF-Luna in)	GPIO 14 / TX	Pin 8	Pi sends config commands to TF-Luna

CRITICAL SETUP STEPS for TF-Luna UART:

- Disable serial console on Pi: sudo raspi-config > Interface > Serial > Login shell: NO > Hardware: YES
- Default TF-Luna baud rate: 115200. Do NOT change unless needed.
- If using I2C mode instead: SDA→Pin 3 (GPIO 2), SCL→Pin 5 (GPIO 3). Default I2C address: 0x10
- Install library: pip3 install PySerial

3.2 Benewake TF02-i — CAN Bus via MCP2515

The TF02-i uses CAN bus natively. Since Raspberry Pi has no built-in CAN, you need an MCP2515 SPI-to-CAN adapter module.

MCP2515 Pin	Pi GPIO	Pi Header #	Notes
VCC	3.3V	Pin 1	MCP2515 module runs on 3.3V
GND	Ground	Pin 9	
SCK	GPIO 11 / SCLK	Pin 23	SPI Clock
SI (MOSI)	GPIO 10 / MOSI	Pin 19	Pi → MCP2515
SO (MISO)	GPIO 9 / MISO	Pin 21	MCP2515 → Pi
CS	GPIO 8 / CE0	Pin 24	Chip select for SPI
INT	GPIO 25	Pin 22	Interrupt line — triggers on new CAN frame

TF02-i CAN wiring (from TF02-i to MCP2515):

- TF02-i CAN-H → MCP2515 CANH terminal (yellow wire on TF02-i)

- TF02-i CAN-L → MCP2515 CANL terminal (green wire on TF02-i)
- TF02-i VCC (12V) → Separate 12V vehicle supply or DC-DC step-up from 5V
- TF02-i GND → Common system GND

Enable SPI + CAN on Pi:

- sudo raspi-config > Interface Options > SPI > Enable
- Add to /boot/config.txt: dtoverlay=mcp2515-can0,oscillator=8000000,interrupt=25
- Add to /boot/config.txt: dtoverlay=spi-bcm2835-overlay
- sudo ip link set can0 up type can bitrate 1000000 (TF02-i default: 1 Mbps)
- Install: sudo apt install can-utils then test: candump can0

3.3 HC-SR04 — Backup GPIO Wiring

HC-SR04 Pin	Pi GPIO	Pi Header #	Notes
VCC	5V	Pin 2	Requires 5V
GND	Ground	Pin 6	
TRIG	GPIO 23	Pin 16	Output from Pi (3.3V is fine for trigger)
ECHO	GPIO 24 (via divider)	Pin 18	MUST use voltage divider: 1kΩ + 2kΩ to step 5V→3.3V to protect Pi

3.4 Full Pin Map Summary

Device	Interface	Pi Pins Used	GPIO Numbers
TF-Luna LiDAR	UART	8, 10	GPIO 14 (TX), GPIO 15 (RX)
MCP2515 (TF02-i CAN)	SPI0	19,21,23,24,22	GPIO 10,9,11,8 (SPI), GPIO 25 (INT)
MPU-6050 IMU	I2C	3, 5	GPIO 2 (SDA), GPIO 3 (SCL) — addr 0x68
HC-SR04 (backup)	GPIO	16, 18	GPIO 23 (TRIG), GPIO 24 (ECHO)
GPS NEO-M8N	UART2*	Use USB adapter	Recommend USB-to-TTL adapter on /dev/ttyUSB0 to avoid UART conflict with TF-Luna
Camera	CSI / USB	CSI ribbon	picamera2 library

NOTE: GPS must use USB-TTL adapter because TF-Luna occupies the Pi's hardware UART (/dev/ttyS0). This is the cleanest solution — GPS appears as /dev/ttyUSB0.

4. Edge Firmware — Sensor Code

4.1 TF-Luna Driver (UART Mode)

```
# tf_luna.py — TF-Luna UART driver for Raspberry Pi
import serial, struct, time

class TFLuna:
    HEADER = 0x59  # TF-Luna frame header byte

    def __init__(self, port='/dev/ttyS0', baudrate=115200):
        self.ser = serial.Serial(port, baudrate, timeout=0.1)
        self.last_valid = None

    def read_distance(self):
        """ Returns distance in cm, or None on error """
        try:
            # Sync to frame start (two consecutive 0x59 bytes)
            while True:
                if self.ser.read(1)[0] == self.HEADER:
                    if self.ser.read(1)[0] == self.HEADER:
                        break
            frame = self.ser.read(7)  # read remaining 7 bytes
            dist_low, dist_high = frame[0], frame[1]
            distance_cm = (dist_high << 8) | dist_low
            # Validate checksum
            checksum = (self.HEADER * 2 + sum(frame[:8])) & 0xFF
            if checksum != frame[8] if len(frame) > 8 else True:
                return None
            self.last_valid = distance_cm
            return distance_cm
        except Exception:
            return None  # Caller will trigger fallback

    def is_healthy(self):
        return self.read_distance() is not None
```

4.2 TF02-i CAN Bus Driver

```
# tf02i_can.py — TF02-i CAN bus driver via MCP2515
import can, struct

# TF02-i default CAN ID for distance frame: 0x700 (check your unit's manual)
TF02I_CAN_ID = 0x700

class TF02iCAN:
    def __init__(self, channel='can0', bitrate=1000000):
        self.bus = can.interface.Bus(channel=channel,
                                     bustype='socketcan')
        self.last_valid = None
```

```
    def read_distance(self):
        """ Returns distance in cm, or None on error """
        try:
            msg = self.bus.recv(timeout=0.05)  # 50ms timeout
            if msg and msg.arbitration_id == TF02I_CAN_ID:
                # TF02-i frame: bytes 0-1 = distance (mm), little-endian
                dist_mm = struct.unpack_from('<H', msg.data, 0)[0]
                dist_cm = dist_mm / 10.0
                self.last_valid = dist_cm
                return dist_cm
        except Exception:
            return None
        return None

    def is_healthy(self):
        return self.read_distance() is not None

    def shutdown(self):
        self.bus.shutdown()
```

4.3 HC-SR04 Backup Ultrasonic Driver

```
# hcsr04.py — HC-SR04 fallback depth sensor
import RPi.GPIO as GPIO, time

TRIG, ECHO = 23, 24
BASELINE_CM = 25.0  # flat road calibration distance (set during install)

GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)

def get_raw_distance():
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)
    timeout = time.time() + 0.04  # 40ms max
    while GPIO.input(ECHO) == 0:
        pulse_start = time.time()
        if time.time() > timeout: return None
    while GPIO.input(ECHO) == 1:
        pulse_end = time.time()
        if time.time() > timeout: return None
    return ((pulse_end - pulse_start) * 34300) / 2

def get_depth_hcsr04():
    raw = get_raw_distance()
    if raw is None: return None
    depth = raw - BASELINE_CM
    return max(0.0, round(depth, 1))
```

4.4 Sensor Fusion Layer — Master Depth Reading

This is the key module that makes the dual-LiDAR + backup system work. It tries TF-Luna first, then TF02-i, fuses both when available, and silently falls back to HC-SR04:

```
# depth_fusion.py — Master sensor fusion depth module
from tf_luna import TFLuna
from tf02i_can import TF02iCAN
from hcsr04 import get_depth_hcsr04
import logging

# Fusion weights: TF-Luna higher weight (better short-range accuracy)
WEIGHT_LUNA = 0.60
WEIGHT_TF02 = 0.40

luna  = TFLuna(port='/dev/ttyS0')
tf02i = TF02iCAN(channel='can0')

def get_fused_depth():
    """
    Returns: (depth_cm: float, source: str)
    source: 'lidar_fused' | 'lidar_luna' | 'lidar_tf02' | 'ultrasonic_backup'
    """
    d_luna = luna.read_distance()
    d_tf02 = tf02i.read_distance()

    if d_luna is not None and d_tf02 is not None:
        # CASE 1: Both LiDARs healthy — weighted fusion
        fused = (WEIGHT_LUNA * d_luna) + (WEIGHT_TF02 * d_tf02)
        return round(fused, 2), 'lidar_fused'

    elif d_luna is not None:
        # CASE 2: Only TF-Luna available
        return round(d_luna, 2), 'lidar_luna'

    elif d_tf02 is not None:
        # CASE 3: Only TF02-i available
        return round(d_tf02, 2), 'lidar_tf02'

    else:
        # CASE 4: Both LiDARs failed — use HC-SR04 backup
        logging.warning('LiDAR offline — switching to HC-SR04 backup')
        depth = get_depth_hcsr04()
        return (depth or 0.0), 'ultrasonic_backup'
```

5. IMU Trigger & Main Orchestration Loop

5.1 Pothole Trigger (MPU-6050)

```
# imu_trigger.py
import smbus2, math, time

MPU_ADDR = 0x68
THRESHOLD_G = 2.5   # tune per vehicle type (bike ~2.5, car ~3.0)
COOLDOWN_S  = 1.5   # ignore re-triggers within 1.5s (same pothole)

bus = smbus2.SMBus(1)
bus.write_byte_data(MPU_ADDR, 0x6B, 0)  # wake up MPU-6050

_last_trigger = 0

def read_accel():
    raw = bus.read_i2c_block_data(MPU_ADDR, 0x3B, 6)
    ax = ((raw[0]<<8)|raw[1]) / 16384.0
    ay = ((raw[2]<<8)|raw[3]) / 16384.0
    az = ((raw[4]<<8)|raw[5]) / 16384.0
    return ax, ay, az

def check_trigger():
    global _last_trigger
    ax, ay, az = read_accel()
    jolt = abs(az - 1.0)  # 1g = normal gravity, subtract baseline
    if jolt > THRESHOLD_G:
        now = time.time()
        if now - _last_trigger > COOLDOWN_S:
            _last_trigger = now
            return True, round(jolt, 3)
    return False, 0.0
```

5.2 Complete Main Loop (Orchestrates Everything)

```
# main.py — Edge device master loop
import paho.mqtt.client as mqtt, json, time, math
from imu_trigger  import check_trigger
from depth_fusion import get_fused_depth
from ai_inference import run_yolo, estimate_width
from gps_reader   import get_gps
import picamera2, cv2

DEVICE_ID    = 'BIKE_001'
MQTT_BROKER  = 'your-cloud.com'
MQTT_TOPIC   = 'smartroad/potholes'
POLL_HZ      = 20  # sensor polling rate
```

```
client = mqtt.Client(DEVICE_ID)
client.connect(MQTT_BROKER, 1883, keepalive=60)
client.loop_start()

cam = picamera2.Picamera2()
cam.start()

def classify_severity(depth_cm, width_cm, g_force):
    score = (depth_cm/5) + (width_cm/20) + (g_force/2)
    return min(5, max(1, int(score)))

print('SmartRoad Edge v2.0 — Dual LiDAR active')
while True:
    triggered, g_force = check_trigger()
    if triggered:
        depth_cm, depth_src = get_fused_depth()
        frame = cam.capture_array()
        bbox  = run_yolo(frame)
        lat, lon = get_gps()

        width_cm = estimate_width(bbox.w_px, depth_cm) if bbox else 0
        severity = classify_severity(depth_cm, width_cm, g_force)

        payload = json.dumps({
            'device_id'  : DEVICE_ID,
            'lat'        : lat,
            'lon'        : lon,
            'depth_cm'   : depth_cm,
            'width_cm'   : width_cm,
            'g_force'    : g_force,
            'severity'   : severity,
            'depth_src'  : depth_src,   #
'lidar_fused'|'lidar_luna'|'lidar_tf02'|'ultrasonic_backup'
            'timestamp'  : time.time()
        })
        client.publish(MQTT_TOPIC, payload, qos=1)

        if depth_src == 'ultrasonic_backup':
            client.publish('smartroad/alerts', json.dumps({
                'device_id': DEVICE_ID, 'alert': 'lidar_offline'}))

    time.sleep(1 / POLL_HZ)
```

6. AI / ML Module

6.1 Model Architecture

Model	Architecture	Input	Output
Pothole Detector	YOLOv8n — fine-tuned on pothole dataset	Camera frame 640×480	Bounding box, confidence, class
Width Estimator	Focal-length formula using fused LiDAR depth	bbox pixel width + depth_cm	Width in cm (much more accurate with LiDAR vs ultrasonic)
Severity Classifier	Weighted rule-based (depth + width + g-force)	3 numeric inputs	Severity level 1–5

6.2 YOLOv8 Training

```
# train_yolov8.py — run on Google Colab (GPU runtime)
!pip install ultralytics roboflow
from ultralytics import YOLO
from roboflow import Roboflow

# Get pothole dataset from Roboflow
rf = Roboflow(api_key='YOUR_API_KEY')
project = rf.workspace().project('pothole-detection')
dataset = project.version(1).download('yolov8')

# Fine-tune YOLOv8 nano (fastest, runs on Pi)
model = YOLO('yolov8n.pt')
model.train(
    data=f'{dataset.location}/data.yaml',
    epochs=100, imgsz=640, batch=16, device='cuda',
    augment=True,    # flip, brightness, blur
    name='pothole_v1'
)
# Export for Raspberry Pi edge inference
model.export(format='tflite')  # optimized for Pi CPU
model.export(format='onnx', opset=12)  # backup runtime
```

6.3 Width Estimation (LiDAR-Enhanced)

Because LiDAR gives ±2mm depth accuracy (vs ±3mm ultrasonic), the width estimate derived from depth is now significantly more reliable:

```
# ai_inference.py
import tflite_runtime.interpreter as tflite
```

```
import cv2, numpy as np

FOCAL_PX     = 600    # calibrate: measure known 30cm object at 100cm
MOUNT_HEIGHT = 40.0   # sensor rig height above road in cm

interpreter = tflite.Interpreter(model_path='pothole_v1.tflite')
interpreter.allocate_tensors()

def run_yolo(frame):
    # Returns bbox object with .w_px, .x1, .y1, .x2, .y2, .conf
    # (implementation uses ultralytics predict with TFLite backend)
    ...

def estimate_width(bbox_pixel_width, lidar_depth_cm):
    """
    LiDAR depth → accurate distance to pothole surface
    Much more accurate than ultrasonic for this formula
    """
    dist_to_surface = MOUNT_HEIGHT - lidar_depth_cm
    if dist_to_surface <= 0: dist_to_surface = 5.0  # safety floor
    width_cm = (bbox_pixel_width * dist_to_surface) / FOCAL_PX
    return round(width_cm, 1)
```

7. Cloud Backend (FastAPI + PostgreSQL)

7.1 API Endpoints

Method	Auth	Route	Description
POST	Device key	/api/potholes/report	Ingest pothole report from edge device or MQTT bridge
GET	Public	/api/potholes?bounds=SW_NE	All potholes in map viewport bounding box
GET	Public	/api/potholes/nearby?lat=&lon=&r=	Potholes within radius (meters) — for driver alerts
PATCH	Govt JWT	/api/potholes/{id}/status	Update repair status: reported / in-repair / repaired
GET	Govt JWT	/api/analytics/summary	KPIs: totals, severity breakdown, sensor source stats
WS	Optional	wss://your-api.com/ws/live	WebSocket: push new potholes to dashboard in real time

7.2 Database Schema — includes depth_src column

```
-- schema.sql (PostgreSQL 15 + PostGIS)
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE potholes (
    id           SERIAL PRIMARY KEY,
    device_id    VARCHAR(50)  NOT NULL,
    location     GEOGRAPHY(POINT, 4326),  -- spatial index-able
    depth_cm     DECIMAL(6,2),
    width_cm     DECIMAL(6,2),
    g_force      DECIMAL(5,3),
    severity     SMALLINT     CHECK(severity BETWEEN 1 AND 5),
    depth_src    VARCHAR(30)  DEFAULT 'lidar_fused',  -- sensor that measured depth
    status       VARCHAR(20)  DEFAULT 'reported',
    image_url    TEXT,
    detected_at  TIMESTAMPTZ  DEFAULT NOW(),
    repaired_at  TIMESTAMPTZ
);

CREATE INDEX potholes_location_gist  ON potholes USING GIST(location);
CREATE INDEX potholes_severity_idx   ON potholes (severity);
CREATE INDEX potholes_depth_src_idx  ON potholes (depth_src);
```

8. Web Dashboard — React + Leaflet

8.1 Dashboard Features

Feature	Description	Role
Live Map	Real-time pothole markers, color-coded by severity (green→purple). LiDAR vs fallback sensor shown on tooltip	Public + Government
Sensor Source Badge	Each marker shows data source: LiDAR Fused / TF-Luna / TF02-i / Ultrasonic Backup. Builds confidence in reading accuracy	Government
Driver Alert (200m)	Browser Notification API fires when user's GPS is within 200m of a pothole. Audio tone + banner.	Public Drivers
Govt Repair Portal	Login-protected: assign contractor, update status, log repair date	Government
Analytics Dashboard	Heatmap, severity chart, sensor reliability pie chart, repair rate KPI tiles	Government
LiDAR Health Monitor	Flag on dashboard when any device switches to ultrasonic backup — prompts field check	Government / Fleet
CSV / PDF Export	Export pothole reports filtered by date, zone, severity, sensor type	Government

8.2 React Map Component — With Sensor Source Display

```
// PotholeMap.jsx
import { MapContainer, TileLayer, CircleMarker, Popup } from 'react-leaflet';
import { useEffect, useState, useRef } from 'react';

const SEV_COLOR = {1:'#27AE60',2:'#F1C40F',3:'#E67E22',4:'#E74C3C',5:'#8E44AD'};
const SRC_LABEL = {
  lidar_fused       : '🟢 LiDAR Fused (TF-Luna + TF02-i)',
  lidar_luna        : '🔵 LiDAR TF-Luna only',
  lidar_tf02        : '🟡 LiDAR TF02-i only',
  ultrasonic_backup : '🔴 ⚠️ Ultrasonic Backup (LiDAR offline)',
};

export default function PotholeMap() {
  const [potholes, setPotholes] = useState([]);
  const [userPos,  setUserPos]  = useState(null);
  const ws = useRef(null);

  useEffect(() => {
    fetch('/api/potholes').then(r=>r.json()).then(setPotholes);
    // Real-time WebSocket feed
    ws.current = new WebSocket('wss://your-api.com/ws/live');
```

```
    ws.current.onmessage = e => {
      const p = JSON.parse(e.data);
      setPotholes(prev => [p, ...prev.slice(0, 999)]);
      checkProximityAlert(p, userPos);  // 200m alert
    };
    // Watch driver GPS
    navigator.geolocation.watchPosition(pos =>
      setUserPos([pos.coords.latitude, pos.coords.longitude]));
  }, []);

  return (
    <MapContainer center={[20.5937, 78.9629]} zoom={13} style={{height:'100vh'}}>
      <TileLayer url='https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png'/>
      {potholes.map(p => (
        <CircleMarker key={p.id} center={[p.lat, p.lon]}
          radius={5 + p.severity * 1.5} color={SEV_COLOR[p.severity]}
          fillOpacity={0.85}>
          <Popup>
            <b>Severity {p.severity}</b><br/>
            Depth: {p.depth_cm} cm | Width: {p.width_cm} cm<br/>
            G-Force: {p.g_force}g<br/>
            Sensor: {SRC_LABEL[p.depth_src]}<br/>
            Status: {p.status}<br/>
            {new Date(p.timestamp*1000).toLocaleString()}
          </Popup>
        </CircleMarker>
      ))}
    </MapContainer>
  );
}
```

9. 12-Week Project Roadmap

Week	Phase	Deliverables	Tools
1	Pi + IMU setup	Pi OS flashed, I2C/SPI/UART enabled, MPU-6050 reading via i2cdetect	Breadboard, jumper wires
2	TF-Luna integration	TF-Luna UART wired, driver code returning distance readings, calibrated	PySerial, oscilloscope (optional)
3	TF02-i CAN setup	MCP2515 wired, CAN0 interface up, candump showing TF02-i frames, Python driver reading	can-utils, MCP2515 module
4	Sensor fusion	depth_fusion.py working: fused mode, fallbacks tested, HC-SR04 backup confirmed	Python, logic analyser
5	GPS + Camera	GPS on USB-TTL delivering coordinates, camera capturing frames at 20fps	gpsd, picamera2
6-7	AI Model Training	YOLOv8n fine-tuned, >85% mAP, TFLite export, running on Pi at ~5fps	Google Colab, Roboflow
8	Full edge integration	main.py running all sensors + AI + MQTT publish in one loop	Mosquitto local broker for testing
9	Cloud backend	FastAPI deployed, PostgreSQL live, MQTT bridge ingesting data	Docker, AWS/GCP/Railway
10	Web dashboard	React app with live map, sensor badges, driver alert, govt portal	React, Leaflet, TailwindCSS
11	Field testing	Mount on bike, drive known pothole roads, validate depth/width vs manual tape measure	Vehicle, calibration tape
12	Polish + deploy	IP67 enclosure waterproofed, systemd auto-start, full end-to-end demo video	3D print/enclosure, heat shrink

10. Sensor Calibration Guide

10.1 TF-Luna Calibration

- Mount sensor pointing straight down at road surface, firmly fixed (no vibration play)
- Park on flat road: record 100 readings at known height. Average = your BASELINE_CM
- Use Benewake TF-Luna GUI tool (Windows) via USB-TTL to verify frame output and change frame rate if needed
- Default 250Hz is ideal — do not reduce below 100Hz for pothole detection
- Valid range check: discard readings < 20cm (too close) or > 600cm (too far = bad reflection)

10.2 TF02-i CAN Calibration

- Use TF02-i config tool or candump to read raw CAN frames first
- TF02-i default CAN ID: 0x700 — confirm with your specific unit's datasheet
- Distance is reported in millimeters in bytes 0–1 (little-endian uint16)
- Park on flat road: verify TF02-i and TF-Luna give matching readings ±5mm
- If deviation > 10mm, check sensor mounting angle — must be perfectly perpendicular to road

10.3 Fusion Validation Test

1. Cover TF-Luna with hand → system should switch to 'lidar_tf02' mode only
2. Cover TF02-i CAN → system should switch to 'lidar_luna' mode only
3. Cover both → system switches to 'ultrasonic_backup', dashboard shows ⚠️ alert
4. Uncover both → system auto-returns to 'lidar_fused' mode within 2 seconds

10.4 Camera Focal Length Calibration

- Print a 30cm × 30cm white square on paper. Tape flat to ground.
- Position sensor rig exactly 100cm above it (use measuring tape).
- Run camera, detect the square. Count pixel width of bounding box.
- FOCAL_PX = (pixel_width × 100) / 30
- Example: if bbox is 200px wide → FOCAL_PX = (200 × 100) / 30 = 667

11. Cloud Deployment (Docker Compose)

```yaml
# docker-compose.yml
version: '3.8'
services:
  mqtt_broker:
    image: eclipse-mosquitto:2
    ports: ['1883:1883', '9001:9001']
    volumes: ['./mosquitto/config:/mosquitto/config']
  backend:
    build: ./backend
    ports: ['8000:8000']
    environment:
      DATABASE_URL: postgresql://user:pass@db:5432/smartroad
      MQTT_BROKER:  mqtt_broker
    depends_on: [db, mqtt_broker]
  db:
    image: postgis/postgis:15-3.3
    environment:
      POSTGRES_DB:       smartroad
      POSTGRES_USER:     smartroad_user
      POSTGRES_PASSWORD: your_secure_password
    volumes: ['pgdata:/var/lib/postgresql/data']
  frontend:
    build: ./frontend
    ports: ['80:80', '443:443']
    depends_on: [backend]
volumes:
  pgdata:
```

11.2 Pi Auto-Start (systemd)

```ini
# /etc/systemd/system/smartroad.service
[Unit]
Description=SmartRoad Pothole Detection Edge v2.0
After=network.target can0.service

[Service]
ExecStartPre=/sbin/ip link set can0 up type can bitrate 1000000
ExecStart=/usr/bin/python3 /home/pi/smartroad/main.py
WorkingDirectory=/home/pi/smartroad
Restart=always
RestartSec=5
User=pi

[Install]
WantedBy=multi-user.target
```

- Enable: sudo systemctl enable smartroad && sudo systemctl start smartroad
- Check logs: sudo journalctl -u smartroad -f

12. Severity Classification & Sensor Comparison

12.1 Severity Levels

Level	Depth	Width	G-Force	Action
1 — Minor	< 2 cm	< 10 cm	< 1.5g	Log only, low-priority maintenance queue
2 — Low	2–5 cm	10–20 cm	1.5–2.5g	Log + green map marker
3 — Mediu m	5–10 cm	20–40 cm	2.5–4g	Alert + government notification
4 — High	10–20 cm	40–60 cm	4–6g	Urgent alert + driver proximity warning
5 — Critical	> 20 cm	> 60 cm	> 6g	Emergency flag, all alerts, immediate escalation

12.2 Depth Sensor Comparison (Your Setup)

Sensor	Accuracy	Update Rate	Range	Role in Your System
TF-Luna (UART)	±2 mm	250 Hz	0.2–8 m	PRIMARY — fast, accurate, ideal for potholes up to 6m deep measurement
TF02-i (CAN)	±1%	100 Hz	0.1–22 m	SECONDARY — long range, industrial grade, fused 40% weight with TF-Luna
HC-SR04 (GPIO)	±3 mm	40 Hz	2 cm–4 m	BACKUP — activates automatically only when both LiDARs are offline
Fused (60/40)	±1–2 mm	100 Hz	0.1–20 m	BEST MODE — highest accuracy, widest range, used for all dimension calculations

Smart Pothole Detection System v2.0 | TF-Luna + TF02-i CAN + HC-SR04 Backup | Full Production Blueprint

Start with Week 1 today: Flash Pi OS, wire MPU-6050, then connect TF-Luna on UART. You'll have live depth readings by Day 3.